

方向 C：面向动力装备的专业知识库与智能体构建

——“鸿雁计划”科创基金细化企划书

汇报人：纪文龙

清华大学行健书院 2024 级本科生

2026 年 4 月

一句话概括

把动力设备领域几十年积累的专家经验“灌”进大模型，建一个能查资料、能推理、能给维修建议的 AI 助手——一线工程师用自然语言问它“这台机器怎么了？”就能得到靠谱答案。发表 1 篇 SCI Q1/Q2 论文。

1 这个研究到底在干什么？（白话版）

1.1 先理解“知识库”是什么

想象一个老师傅在发电厂工作了 30 年，他脑子里装着海量的经验：哪种振动波形对应轴承磨损、什么温度曲线意味着快要出事、遇到这种故障应该先查哪个部件……这些经验就是**领域知识**。

问题是：老师傅会退休、会离职，经验带走了就没了。而且一个人的经验再丰富也有盲区。**知识库**就是把这些分散在技术文档、维修记录、行业标准、学术论文里的知识**数字化、结构化**，存进计算机里，让任何人都能检索和使用。

1.2 那“智能体”是什么？

你可能已经用过 ChatGPT——它能聊天、能写文章，但它**不懂动力设备**。你问它“这台汽轮机的 2X 频振动偏高可能是什么原因？”它可能瞎编一个看似合理但实际错误的答案。这叫**幻觉（hallucination）**。

智能体（Agent） = 大模型 + 工具调用 + 知识检索。它不是光靠“脑子里”的知识回答，而是先去**知识库**里找相关资料，再结合资料进行推理。就好比一个医生不光靠记忆诊断，还要查检验报告、翻教科书——这样的诊断才靠谱。

1.3 这两者结合起来能干什么？

- 故障诊断辅助**：一线工程师描述现象（“3 号机组振动突然升高”），智能体检索类似历史案例、查阅设备手册，给出可能的故障原因和排查建议。
- 知识问答**：新入职的工程师不用翻几百页的技术手册，直接问智能体。
- 经验沉淀**：老专家的诊断记录、维修日志不再是“死”的文档，而是变成智能体可以检索和推理的“活”知识。

1.4 所以我们要做的核心事情是：

针对动力设备（汽轮机、发电机、轴承等）场景，（1）构建一个专业知识库——把技术文档、故障案例、运行规律整理成大模型能“读懂”的格式；（2）在知识库上搭建一个 RAG 智能体——让它在回答问题时先检索知识库再生成答案，消除幻觉、保证专业性。

2 为什么现在做这件事正当时？

2.1 大模型能力已经成熟，但工业落地还有“最后一公里”

2023–2025 年，GPT-4、Claude、Qwen 等大模型的通用能力已经很强了。但工业界的反馈是：通用大模型直接用到专业领域不靠谱——回答不够精准、容易编造、缺乏可追溯性。

2.2 RAG（检索增强生成）解决了“幻觉”问题

RAG 的核心思想很简单：模型回答问题之前，先从一个可信的知识库中检索出相关段落，把这些段落作为“参考资料”附在提示词里，然后让模型基于这些资料来生成答案。这样模型的回答就有据可查，不再是“空口说白话”。

技术上包括三步：

1. **索引（Indexing）**：把文档切成小段（chunk），用 Embedding 模型把每段文字变成一个向量（一串数字），存进向量数据库。
2. **检索（Retrieval）**：用户提问也转成向量，找到知识库中语义最接近的几段。
3. **生成（Generation）**：把检索到的段落和用户问题一起交给大模型，生成最终答案。

2.3 从“基础 RAG”到“Agentic RAG”——智能体化

基础 RAG 是“被动的”：用户问什么就搜什么。但真实的故障诊断需要**多步推理**：先确定症状 → 缩小可能原因 → 逐步排查 → 给出建议。这就需要有一个 **Agent 框架** 让大模型能够“思考、规划、调用工具”。

当前文献空白：大模型 + RAG 的工业应用论文很多，但针对能源动力设备（汽轮机、发电机组等）的系统性知识库和智能体工作还很少。刘老师课题组在故障诊断领域积累了大量数据和方法论，正好可以填补这个空白。

3 我们的方法：PowerKB-Agent

3.1 整体架构（用大白话讲）

我们的系统叫 **PowerKB-Agent**（Power equipment Knowledge Base Agent），分三层：

1. **知识层（Knowledge Layer）**：

- 收集动力设备的技术文档（设备手册、国标、行业规范）、故障案例库（历史维修记录、典型故障分析报告）、学术论文（刘老师课题组发表的高被引论文等）。
- 对文档进行清洗-分块-标注：清洗掉无关内容（页眉页脚、广告等），按语义切成合理大小的文本块，给每个块打上元数据标签（设备类型、故障类别、知识类型等）。
- 用 Embedding 模型（如 BGE-large-zh）将文本块编码为向量，存入向量数据库（Milvus 或 Chroma）。

2. 检索层（Retrieval Layer）：

- 用户提问 → 转成向量 → 在向量数据库中找到 Top-K 最相关的文本块。
- 加入重排序器（Reranker）：粗检索找回 20 个候选，再用一个小模型精排，选出最相关的 5 个——提高检索精度。
- 支持混合检索：向量相似度 + 关键词匹配（BM25），两种方式互补，确保既能找到语义相近的，也能找到术语完全匹配的。

3. 智能体层（Agent Layer）：

- 基于 LangChain/LlamaIndex 搭建 Agent 框架。
- Agent 能做的事：（a）检索知识库；（b）调用故障诊断规则（if-then 专家规则）；（c）查询结构化数据库（设备参数表、历史运行数据）；（d）多轮对话追问用户以获取更多信息。
- 最终输出带来源引用的回答——每个诊断建议都标注“这条信息来自 XX 文档第 XX 页”，可追溯、可验证。

3.2 为什么这样设计有效？

- 知识库保证专业性：答案基于真实的技术文档和案例，不是大模型“编”的。
- RAG 消除幻觉：有检索做支撑，大模型不会胡说。
- Agent 实现多步推理：不是简单的一问一答，而是能分步骤思考和排查。
- 引用溯源增强信任：工程师可以直接点开引用来源核实，符合工业场景的可靠性要求。

3.3 三层架构与工作量

1. 底层·数据：课题组积累的动力设备故障案例库、技术文档、公开的国标/行标、刘老师团队发表的论文——这是我们相对于其他团队的**独家优势**。
2. 中层·系统：RAG Pipeline + Agent 框架——这是论文的核心贡献，重点在知识库构建方法论和检索增强策略。
3. 上层·展示：一个对话式 Web 界面，输入故障描述就能得到诊断建议和引用来源。

4 实验设计（怎么证明系统有效？）

4.1 评估思路

和方向 A/B 不同，这个方向不是”跑神经网络算准确率”，而是评估一个问答系统的质量。我们需要回答两个问题：

- **检索质量**：给一个问题，系统找到的文档段落是否确实包含正确答案？
- **生成质量**：基于检索到的段落，模型最终给出的回答是否准确、完整、有用？

4.2 评估数据集构建

我们需要人工构建一个**测试题库**（约 100-200 道题），来源包括：

- 从课题组的故障案例中提取真实问答对（”问：某机组出现 XX 现象，可能原因？答：……”）
- 从课题组论文中提取技术性问题（”问：DANN 中梯度反转层的作用是什么？”）
- 从设备手册中提取操作性问题（”问：XX 型轴承的允许温升范围？”）

4.3 评价指标

指标	含义	通俗解释
Hit Rate@K	检索回来的 Top-K 段落中是否包含正确答案	”前 5 个搜索结果里有没有答案”
MRR	正确答案在检索结果中的平均排名位置	”答案一般排第几？越靠前越好”
Faithfulness	生成的回答是否与引用的段落内容一致	”模型有没有偏离引用的资料”
Answer Relevancy	回答对问题的相关程度	”有没有答非所问”
专家评分	邀请刘老师/课题组博士生评分	”行内人觉得这个回答靠谱吗”

4.4 消融实验

逐步去掉系统的组件，看性能变化：

- 去掉 RAG（= 纯大模型直接回答）→ 证明知识库检索的价值
- 去掉 Reranker（= 只用粗检索）→ 证明重排序的价值
- 去掉混合检索（= 只用向量检索）→ 证明关键词匹配的互补作用
- 去掉 Agent（= 单轮 RAG）→ 证明多步推理的价值
- 去掉元数据过滤（= 不按设备/故障类型筛选）→ 证明结构化标注的价值

4.5 对比基线

- **纯大模型**：直接用 GPT-4/Qwen 回答专业问题（不检索）
- **基础 RAG**：Naive RAG（固定大小分块 + 纯向量检索 + 单轮生成）

- **Google 搜索：**用搜索引擎搜同样的问题，看返回结果质量
- **传统专家系统：**基于 if-then 规则的故障诊断系统

只要 PowerKB-Agent 在 Faithfulness 和专家评分上明显优于上述基线，论文就有说服力。

5 预期成果

- **1 篇 SCI Q1/Q2 论文：**投向 *Expert Systems with Applications* (IF~8.5) 或 *Knowledge-Based Systems* (IF~8.1)。“领域知识库 +RAG+Agent 应用于能源动力设备智能运维”是当前的热点空白。
- **动力装备知识库：**一个结构化、可检索的专业知识库（含文档 >500 篇、标注问答对 >200 组），可供课题组长期使用和迭代。
- **智能体演示系统：**一个 Web 界面的对话式诊断助手，输入故障描述即可获得带引用溯源的诊断建议。
- **鸿雁计划结题：**完整结题报告 + 上述成果。

6 W1–W30 逐周工作计划

每周投入 8 小时，分 6 阶段。

6.1 阶段一：调研 + 认识领域（W1–W5，4 月）

这个阶段的目标是：搞懂大模型 +RAG+Agent 在工业领域的应用现状，同时摸清课题组有哪些可用的知识资料。

- W1** 读 20 篇相关论文。搜索“LLM industrial fault diagnosis”“RAG knowledge base power equipment”“Agentic RAG maintenance”，做方法对比表（架构/知识来源/评估方式/效果/年份）。重点关注：别人的知识库怎么建的？RAG 的检索策略用了什么？Agent 的工具调用设计了哪些？
- W2** 精读 3–5 篇核心论文。(1) RAG 综述（最新一篇 Survey）；(2) 工业领域 Agent 应用论文；(3) 刘老师发表的故障诊断方法论文（理解课题组的核心技术体系）。输出：一份 2 页的技术选型报告发刘老师确认（用哪个大模型、哪个向量数据库、哪个 Agent 框架）。
- W3** 盘点课题组知识资源。和课题组博士生沟通，梳理可用的文档资料：设备技术手册有哪些？历史故障案例库存在什么地方？格式是什么（PDF/Word/数据库记录）？整理一份“知识资源清单”。
- W4** 学习工具链。动手跑通一个最简单的 RAG Demo：用 LangChain 加载几份 PDF 文档 → 切分 → 存入 Chroma 向量数据库 → 问几个问题看效果。目的是把整个 Pipeline 的每一步都亲手走一遍。
- W5** 学习 Agent 框架。在 W4 的基础上给 Agent 加“工具”：比如一个能查设备参数表的工具、一个能搜索故障案例的工具。确认自己能用 LangChain/LlamaIndex 搭出一个带工具调用的 Agent。

6.2 阶段二：知识库构建（W6-W10, 5-6 月）

这个阶段的目标是：把课题组的知识资料真正地”灌”进系统里，建成一个可检索的专业知识库。

- W6 文档预处理 Pipeline。**开发一套自动化脚本：(1) PDF/Word → 纯文本（用 PyMuPDF 或 Unstructured 库处理排版、表格、公式）；(2) 文本清洗（去页眉页脚、修复乱码）；(3) 智能分块（按段落/章节语义切分，而非固定字数切），每块附加元数据（文档标题、章节名、设备类型、日期）。先处理 50 份文档，验证 Pipeline 可用性。
- W7 大规模文档编码入库。**用 BGE-large-zh（或其他中文 Embedding 模型）对所有文本块生成向量，存入向量数据库。同时建 BM25 倒排索引用于关键词检索。目标：处理完全部可用文档（预计 300-500 份）。
- W8 知识标注与质量检查。**(1) 对重要文档块人工标注”设备类型-故障模式-知识类型”三级标签，支持后续按类别过滤检索；(2) 从知识库中随机抽 50 个问题做初步检索测试，看 Top-5 命中率。
- W9 构建评估题库。**从故障案例和论文中人工编写 100-200 道测试问题（含标准答案和引用来源），包括：事实型（”XX 设备的额定参数是？”）、诊断型（”出现 XX 现象可能是什么故障？”）、方法型（”如何用 DANN 做跨域迁移？”）三类。
- W10 阶段验证 + 汇报。**用测试题库评估”基础 RAG”的效果——Hit Rate@5、MRR 等指标。把结果发刘老师，确认知识库基础质量合格后进入下一阶段。

6.3 阶段三：核心研发——RAG 优化与 Agent 搭建（W11-W15, 7 月）

这是最关键的阶段：从”能跑”提升到”跑得好”，实现论文的核心创新。

- W11 检索优化。**(1) 实现混合检索（向量 + BM25 加权融合）；(2) 加入 Reranker 模型（bge-reranker-large）做二次精排；(3) 实现 Query Expansion——把用户短问题用 LLM 扩展成更详细的检索 Query（比如”振动高”→”旋转机械振动幅值偏高可能原因排查”），提升召回率。
- W12 搭建 Agent 框架。**定义 Agent 可调用的工具集：(1) 知识库向量检索工具；(2) 设备参数结构化查询工具（SQL 查设备型号/参数表）；(3) 故障规则推理工具（基于课题组总结的专家规则）；(4) 追问工具（当信息不足时主动向用户追问）。用 LangGraph 或 AutoGen 实现 Agent 的”规划-执行-反思”循环。
- W13 Prompt 工程。**精心设计 System Prompt：让智能体扮演”动力设备高级诊断工程师”角色，明确回答格式要求（必须列出故障可能性、排查建议、引用来源），设置安全规则（不确定时明确说”我不确定”而非编造）。测试不同 Prompt 对回答质量的影响。
- W14 端到端联调。**把知识层、检索层、Agent 层串联起来，用评估题库跑完整的端到端测试。记录全部指标，找出短板（是检索不准？还是生成不好？还是 Agent 逻辑有问题？）。
- W15 调优 + 向刘老师汇报。**针对短板逐一优化（调分块大小、检索 Top-K、Reranker 阈值、Prompt 措辞等），写中期报告。

6.4 阶段四：大规模实验——出论文数据（W16-W20，8-9 月）

这个阶段的目标是：跑完所有对比实验和消融实验，产出论文所需的全部数据和图表。

- W16 消融实验。**5 种消融配置（纯 LLM / 无 Reranker / 无混合检索 / 无 Agent / 无元数据过滤）× 完整评估题库，记录所有指标对比。
- W17 基线对比实验。**跑 4 种基线（纯 GPT-4、Naive RAG、Google 搜索、传统专家系统）在同一题库上的表现，做完整对比表。
- W18 专家评估。**邀请刘老师和 2-3 位课题组博士生对 50 道诊断型问题的回答做盲评打分（1-5 分的准确性、完整性、实用性），计算评分间一致性（Cohen's Kappa）。
- W19 案例分析。**选 5 个典型故障诊断场景，展示 Agent 的完整推理过程（用户提问 → Agent 规划检索 → 获取信息 → 逐步推理 → 给出建议），做成论文中的 Case Study 图。
- W20 结果整理。**汇总全部实验数据，画论文图表：性能对比柱状图、消融热力图、Agent 推理流程图、知识库统计信息表等。

6.5 阶段五：写论文 + 搭演示系统（W21-W25，10-11 月）

- W21 搭建 Web 演示界面。**用 Streamlit 或 Gradio 做一个对话式界面：左侧输入故障描述、右侧显示智能体回答（含引用来源链接），底部显示 Agent 的”思考过程”（检索了什么、调用了什么工具）。
- W22 和刘老师确认论文结构和目标期刊；**写论文第 1 章（动力设备智能运维的背景和需求、为什么需要领域知识库 + 智能体）。
- W23 写论文第 2 章（相关工作：RAG 技术综述、工业领域 LLM 应用现状、知识库构建方法论）。**
- W24 写论文第 3 章（PowerKB-Agent 系统设计：知识库构建流程、检索增强策略、Agent 架构，含架构图和算法伪代码）。**
- W25 写论文第 4 章（实验结果：全部对比表、消融分析、案例分析、专家评估结果）。**

6.6 阶段六：投稿 + 结题（W26-W30，12-1 月）

- W26 第 5 章（结论、局限性讨论、未来展望）+ 全文润色。**
- W27 给刘老师审改，根据反馈修改补充。**
- W28 按目标期刊模板排版，提交投稿。**
- W29 写鸿雁计划结题报告（约 15 页）。**
- W30 提交结题材料包，项目完成。**